CrownCare Firebase Analytics Schema

Purpose: Master analytics tracking for CrownCare app across iOS and Android platforms. Captures user behavior, subscription conversions, feature usage, and revenue metrics.

Database: Firebase Realtime Database (you can migrate to Firestore later if needed)

1. Database Structure Overview

```
/analytics/ /events/ /[yyyy-mm-dd]/ /[event-id]/ { event data } /users/ /[user-id]/ { user profile } /sessions/ /[session-id]/ { session data }
```

Why this structure?

- Events organized by date makes it easy to query "all events on May 3rd"
- Users indexed by user-id for retention and cohort analysis
- Sessions for calculating session duration and user flow

2. Core Event Schema

Every event emitted from the app follows this structure:

```
{ "event_id": "uuid", "event_type": "string", "user_id": "string", "platform": "ios" | "android", "app_version": "string", "session_id": "string", "timestamp": 1714752000000, "timestamp_iso": "2026-05-03T14:00:00Z", "properties": { } }
```

`event_id`
`UUID (string)`
Unique identifier for each event. Generate client-side to prevent duplicates on retry.

`event_type`
`string`

Category of event: "app_open", "paywall_shown", "subscription_completed", "feature_view", etc.

user_id
`string`
Anonymous ID before subscription (Firebase Auth UID or UUID), identified after signup/subscription completion.

platform
`"ios" | "android"`
Source platform. Required for platform-specific analytics and breakdowns.

app_version
`string (e.g., "1.0.0")`
Helps identify if bugs/behavior changes correlate with releases.

session_id
`UUID (string)`
Groups events into sessions. Generate new session_id on app_open.

timestamp
`number (milliseconds since epoch)`
Unix timestamp for easy sorting and queries.

timestamp_iso
`string`
Human-readable ISO 8601 format for debugging and reports.

properties
`object`
Event-specific data. See below for each event type.

3. Event Types & Properties

app_open

Fired when user opens the app (used for DAU calculation).

```
"properties": { "session_duration_ms": 0, "is_returning_user": true,
"days_since_last_open": 5 }
```

paywall_shown

When the subscription paywall is displayed.

```
"properties": { "location": "string", // "onboarding" | "settings" |
"feature_lock" "tiers_shown": ["basic", "premium"] }
```

subscription_initiated

When user taps a subscription tier (begins purchase flow).

```
"properties": { "tier": "basic" | "premium", "tier_price_usd": 9.99,
"billing_period": "monthly" | "annual" }
```

subscription_completed

When subscription purchase succeeds (from App Store/Play Store webhook receipt).

```
"properties": { "tier": "basic" | "premium", "tier_price_usd": 9.99,
"billing_period": "monthly" | "annual", "transaction_id": "string",
"receipt_data": "base64_encoded_receipt", "converted_from_trial": false,
"days_to_conversion": 3 }
```

feature_view

When user views a major feature (hairstyles, tools, categories).

```
"properties": { "feature_name": "string", // Examples: "hairstyle_gallery",
"styling_tools", "color_mixer" "category": "string", // If applicable
"time_spent_seconds": 45 }
```

feature_interaction

When user performs an action within a feature.

```
"properties": { "feature_name": "string", "action": "string", // e.g.,
"save_hairstyle", "share_image" "success": true, "error_message": null }
```

subscription_cancel

When user cancels their active subscription.

```
"properties": { "tier": "premium", "days_subscribed": 45, "reason": "string", // Optional user feedback "churn_category": "price_sensitive" | "feature_gap" | "other" }
```

app_crash

When app crashes (implement in crash handler).

```
"properties": { "error_message": "string", "stack_trace": "string", "memory_available_mb": 256 }
```

app_background

When user leaves app (use to calculate session_duration).

```
"properties": { "session_duration_ms": 45000, "features_accessed_count": 3 }
```

4. User Profile Schema

Stored at `/analytics/users/[user-id]/`

```
{ "user_id": "uuid", "created_at": 1714752000000, "first_platform": "ios" | "android", "platforms": ["ios", "android"], "subscription_status": "free" | "basic" | "premium" | "cancelled", "current_tier": "basic" | "premium" | null, "tier_start_date": 1714752000000, "total_subscriptions": 1, "lifetime_revenue_usd": 9.99, "last_seen": 1714752000000, "session_count": 42, "total_session_duration_ms": 18000000 }
```

5. Session Schema

Stored at `/analytics/sessions/[session-id]/`

```
{ "session_id": "uuid", "user_id": "uuid", "platform": "ios", "start_time": 1714752000000, "end_time": 1714752045000, "duration_ms": 45000, "event_count": 12, "features_accessed": ["hairstyle_gallery",
```

```
"styling_tools"], "paywall_shown": true,
"subscription_completed_in_session": false }
```

6. Dashboard Query Patterns

These are the queries your master dashboard will run:

Metric	Query Logic	Frequency
Daily Active Users (DAU)	COUNT(DISTINCT user_id) WHERE event_type="app_open" AND date=[today]	Daily
Monthly Active Users (MAU)	COUNT(DISTINCT user_id) WHERE event_type="app_open" AND date >= [30 days ago]	Daily
Conversion Rate	COUNT(subscription_completed) / COUNT(paywall_shown) * 100	Real-time
Revenue by Platform	SUM(tier_price_usd) WHERE event_type="subscription_completed" GROUP BY platform	Real-time
Subscription Breakdown	COUNT(subscription_status) GROUP BY tier, platform	Hourly
Churn Rate	COUNT(subscription_cancel) / COUNT(total_active) * 100	Daily
Feature Popularity	COUNT(feature_view) GROUP BY feature_name ORDER BY count DESC	Daily
Retention Day-N	COUNT(DISTINCT user_id WHERE last_seen >= [N days since first_open]) / COUNT(cohort_size) * 100	Daily

7. Data Organization by Date

Events are organized by date for efficient querying:

```
/analytics/events/2026-05-03/ /[event-id-1]/ /[event-id-2]/ /[event-id-3]/
/analytics/events/2026-05-04/ /[event-id-4]/
```

Why: Querying "give me all events from May 3rd" is one direct path lookup, not scanning the entire events tree.

8. User Identification Strategy

> **Before subscription:** Use Firebase Authentication UID or generate a persistent UUID stored locally
>
> **After subscription:** Link the anonymous ID to their account/subscription record so you can track the same user pre- and post-conversion

9. Data Retention Policy

- **Raw events:** Keep for 12 months (cost: ~500MB/month of events)
- **User profiles:** Keep indefinitely (small dataset)
- **Sessions:** Archive after 90 days (can aggregate into daily summaries)
- **Archive strategy:** At end of each month, export events to Google Cloud Storage, then delete from Realtime Database

10. Implementation Checklist for iOS App

When you build the event instrumentation code, you'll need:

- Analytics service class that wraps Firebase
- Session ID generation on app launch
- App lifecycle hooks (AppDelegate: willFinishLaunchingWithOptions, applicationDidEnterBackground)
- Paywall listeners to fire paywall_shown events
- StoreKit 2 receipt observer to fire subscription_completed

- Queue + retry logic for offline event submission
- Device UUID persisted locally for anonymous users

11. Firebase Rules (Security)

> **Important:** Your app should only write to its own user's data and events. Set Firebase rules to prevent users from writing arbitrary data.

```
{ "rules": { "analytics": { "events": { ".read": false, ".write": true // Lock this down in production }, "users": { "$user_id": { ".read": "auth.uid == $user_id || root.child('admins').child(auth.uid).exists()", ".write": "auth.uid == $user_id" } } } } }
```

Summary

This schema gives you:

- ✓ Complete user journey tracking (free → paywall → subscription)
- ✓ Platform-specific breakdowns (iOS vs Android)
- ✓ Feature usage insights (which hairstyles, tools are popular)
- ✓ Revenue and conversion metrics
- ✓ Retention and churn analysis
- ✓ Efficient date-based queries for dashboards

Once you approve this schema, the next step is building the iOS Analytics service class that implements these event types.